머클 트리와 상태 증명

개요

머클 트리(Merkle Tree)는 CometBFT에서 데이터 무결성을 보장하고 효율적인 증명을 제공하는 핵심 암호학적 데이터 구조입니다. CometBFT는 RFC-6962 표준을 따르는 바이너리 머클 트리를 구현하여 트랜잭션 결과, 블록 데이터, 그리고 애플리케이션 상태의 무결성을 검증합니다.

핵심 개념

머클 트리의 정의

머클 트리는 다음과 같은 특성을 가진 바이너리 트리입니다:

- **리프 노드**: 실제 데이터의 해시값을 저장
- **내부 노드**: 자식 노드들의 해시값을 결합한 해시값을 저장
- **루트 노드**: 전체 데이터 세트를 대표하는 단일 해시값
- **결정성**: 동일한 데이터 세트는 항상 동일한 루트 해시를 생성

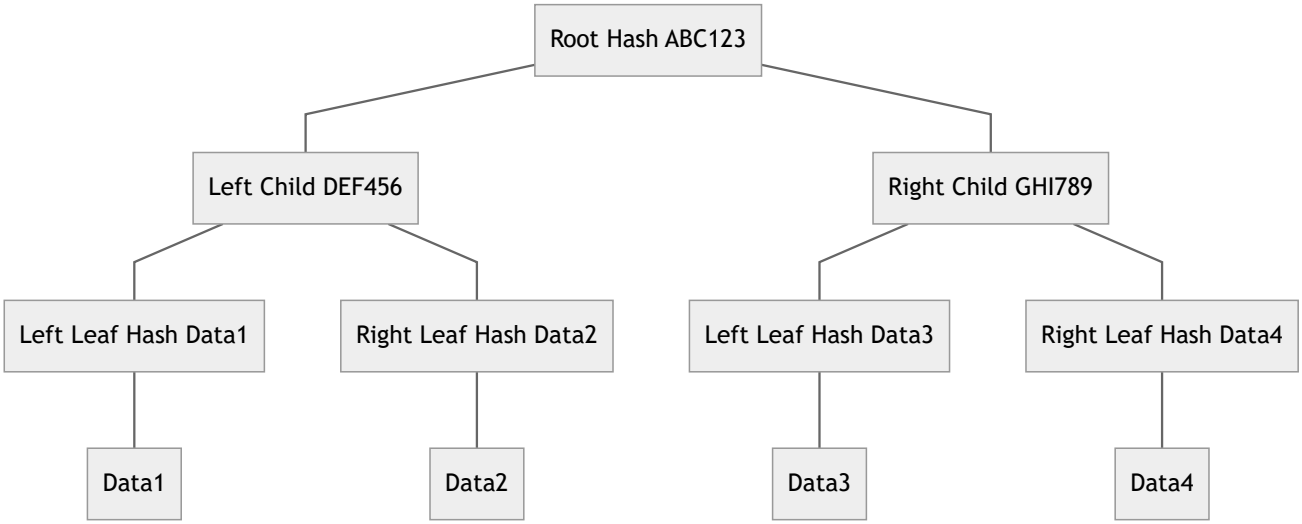
상태 증명의 목적

상태 증명(State Proof)은 다음과 같은 용도로 사용됩니다:

- **데이터 무결성 검증**: 특정 데이터가 변조되지 않았음을 증명
- **부분 검증**: 전체 데이터를 다운로드하지 않고도 특정 항목의 존재를 증명
- **라이트 클라이언트 지원**: 최소한의 데이터로 블록체인 상태 검증
- **효율적인 동기화**: 상태 차이만을 전송하여 빠른 동기화 지원

아키텍처

CometBFT 머클 트리 구조



해시 함수 구현

CometBFT는 SHA-256을 기반으로 한 커스텀 해시 함수를 사용합니다:

```
// 리프 노드 해시: SHA256(0x00 || data)
func leafHash(leaf []byte) []byte {
return tmhash.Sum(append(leafPrefix, leaf...))
}

// 내부 노드 해시: SHA256(0x01 || left || right)
func innerHash(left []byte, right []byte) []byte {
return tmhash.SumMany(innerPrefix, left, right)
```

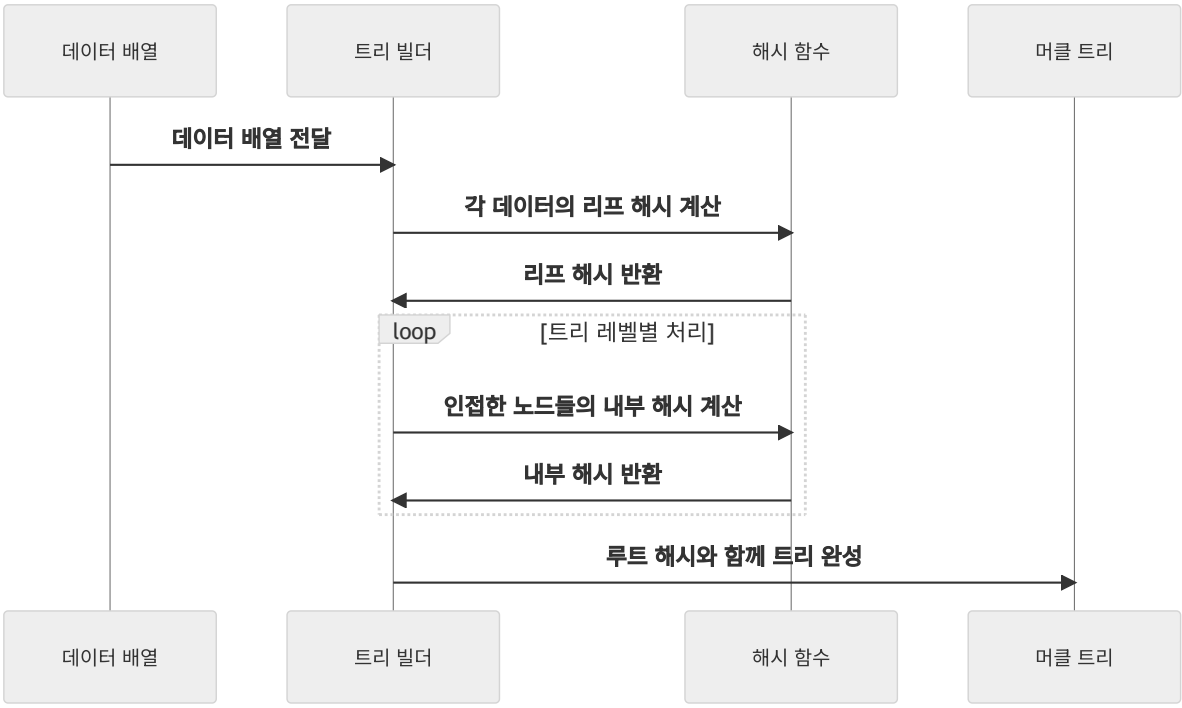
```
}
```

프리픽스 사용 이유

- **0x00 (리프 프리픽스)**: 리프 노드임을 명시하여 second preimage attack 방지
- **0x01 (내부 프리픽스)**: 내부 노드임을 명시하여 구조적 무결성 보장

동작 과정

머클 트리 생성 과정



증명 생성 과정

특정 데이터에 대한 머클 증명을 생성하는 과정:

1. **타겟 데이터 위치 확인**: 증명하려는 데이터의 인덱스 파악
2. **경로 추적**: 리프에서 루트까지의 경로 계산
3. **Aunt 노드 수집**: 경로상의 각 레벨에서 형제 노드(aunt) 해시 수집
4. **증명 구조체 생성**: 수집된 정보로 Proof 객체 생성

코드 분석

머클 트리 생성 구현

```
func HashFromByteSlices(items [][]byte) []byte {
return hashFromByteSlices(sha256.New(), items)
}

func hashFromByteSlices(sha hash.Hash, items [][]byte) []byte {
switch len(items) {
case 0:
return emptyHash()
case 1:
return leafHashOpt(sha, items[0])
default:
k := getSplitPoint(int64(len(items)))
left := hashFromByteSlices(sha, items[:k])
right := hashFromByteSlices(sha, items[k:])
return innerHashOpt(sha, left, right)
```

```
}
}
```

분할점 계산

```
func getSplitPoint(length int64) int64 {
if length < 1 {
panic("Trying to split a tree with size < 1")
}
uLength := uint(length)
bitlen := bits.Len(uLength)
k := int64(1 << uint(bitlen-1))
if k == length {
k >>= 1
}
return k
}
```

이 함수는 트리를 균형있게 분할하기 위해 가장 큰 2의 거듭제곱 값을 찾습니다.

증명 구조체

```
type Proof struct {
Total int64 `json:"total"` // 전체 항목 수
```

```
Index int64 `json:"index"` // 증명할 항목의 인덱스
LeafHash []byte `json:"leaf_hash"` // 항목의 리프 해시
Aunts [][]byte `json:"aunts,omitempty"` // 루트까지의 경로상 형제 노드들
}
```

증명 검증 구현

```
func (sp *Proof) Verify(rootHash []byte, leaf []byte) error {
if rootHash == nil {
return ErrInvalidHash{Err: errors.New("nil root")}
}
// 리프 해시 검증
hash := tmhash.New()
leafHash := leafHashOpt(hash, leaf)
if !bytes.Equal(sp.LeafHash, leafHash) {
return ErrInvalidHash{
Err: fmt.Errorf("leaf %x, want %x", sp.LeafHash, leafHash),
}
}
// 루트 해시 계산 및 검증
computedHash, err := sp.computeRootHash(hash)
if err != nil {
return ErrInvalidHash{Err: fmt.Errorf("compute root hash: %w", err)}
```

```
}
if !bytes.Equal(computedHash, rootHash) {
return ErrInvalidHash{
Err: fmt.Errorf("root %x, want %x", computedHash, rootHash),
}
}
return nil
}
```

루트 해시 계산

```
func computeHashFromAunts(hash hash.Hash, index, total int64, leafHash []byte,
if index >= total || index < 0 || total <= 0 {
return nil, fmt.Errorf("invalid index %d and/or total %d", index, total)
}
switch total {
case 0:
panic("Cannot call computeHashFromAunts() with 0 total")
case 1:
if len(innerHashes) != 0 {
return nil, errors.New("unexpected inner hashes")
}
return leafHash, nil
```

```go
default:
if len(innerHashes) == 0 {
return nil, errors.New("expected at least one inner hash")
}
numLeft := getSplitPoint(total)
if index < numLeft {
// 왼쪽 서브트리에 위치
leftHash, err := computeHashFromAunts(hash, index, numLeft, leafHash, innerHashe
if err != nil {
return nil, err
}
return innerHashOpt(hash, leftHash, innerHashes[len(innerHashes)-1]), nil
}
// 오른쪽 서브트리에 위치
rightHash, err := computeHashFromAunts(hash, index-numLeft, total-numLeft, leafH
if err != nil {
return nil, err
}
return innerHashOpt(hash, innerHashes[len(innerHashes)-1], rightHash), nil
}
}
```

실제 사용 사례

트랜잭션 결과 증명

```go
type ABCIResults []*abci.ExecTxResult

// 모든 결과의 머클 해시 계산
func (a ABCIResults) Hash() []byte {
return  merkle.HashFromByteSlices(a.toByteSlices())
}

// 특정 결과에 대한 머클 증명 생성
func (a ABCIResults) ProveResult(i  int) merkle.Proof {
_, proofs := merkle.ProofsFromByteSlices(a.toByteSlices())
return *proofs[i]
}
```

블록 헤더에서의 활용

블록 헤더는 다음과 같은 머클 루트들을 포함합니다:

```go
type Header struct {
// ... 다른 필드들
DataHash []byte  // 트랜잭션들의 머클 루트
ValidatorsHash []byte  // 밸리데이터 세트의 머클 루트
```

```
NextValidatorsHash []byte  // 다음 밸리데이터 세트의 머클 루트
ConsensusHash []byte  // 합의 파라미터의 해시
AppHash []byte  // 애플리케이션 상태의 루트 해시
LastResultsHash []byte  // 이전 블록 실행 결과의 머클 루트
// ... 다른 필드들
}
```

설정 및 튜닝

성능 최적화

반복적 구현 vs 재귀적 구현

CometBFT는 두 가지 구현을 제공합니다:

```
// 재귀적 구현 (기본)
func  HashFromByteSlices(items [][]byte) []byte {
return  hashFromByteSlices(sha256.New(), items)
}

// 반복적 구현 (스택 오버플로우 방지)
func  HashFromByteSlicesIterative(input [][]byte) []byte {
items := make([][]byte, len(input))
sha := sha256.New()
// 초기 리프 해시 계산
```

```go
for i, leaf := range input {
items[i] = leafHash(leaf)
}
// 상향식으로 트리 구성
size := len(items)
for {
switch size {
case 0:
return emptyHash()
case 1:
return items[0]
default:
rp := 0 // 읽기 위치
wp := 0 // 쓰기 위치
for rp < size {
if rp+1 < size {
items[wp] = innerHashOpt(sha, items[rp], items[rp+1])
rp += 2
} else {
items[wp] = items[rp]
rp++
}
wp++
```

```
}
size = wp
}
}
}
```

보안 설정

최대 Aunt 수 제한

```
const MaxAunts = 100

func (sp *Proof) ValidateBasic() error {
if len(sp.Aunts) > MaxAunts {
return ErrMaxAuntsLenExceeded
}
// ... 다른 검증 로직
}
```

이 제한은 DoS 공격을 방지하기 위해 설정되었습니다.

문제 해결

일반적인 머클 트리 관련 이슈

1. 증명 검증 실패

```
Error: merkle: invalid proof: leaf hash mismatch
```

해결 방법:

- 원본 데이터와 증명에 사용된 데이터가 일치하는지 확인
- 리프 해시 계산 과정에서 프리픽스가 올바르게 적용되었는지 검증
- 데이터 직렬화 방식의 일관성 확인

2. 루트 해시 불일치

```
Error: merkle: invalid hash: root ABC123, want DEF456
```

해결 방법:

- 트리 구성 순서가 올바른지 확인
- 분할점 계산 로직 검증
- 해시 함수 구현의 일관성 확인

3. Aunt 배열 크기 오류

```
Error: merkle: maximum aunts length, 100, exceeded
```

해결 방법:

- 데이터 세트 크기 확인 (2^100 이하여야 함)
- 증명 생성 과정에서 불필요한 aunt 노드 제거
- 데이터 분할 전략 재검토

디버깅 도구

머클 트리 시각화

```
func (sp *Proof) StringIndented(indent  string) string {
return  fmt.Sprintf(`Proof{
%s Total: %d
%s Index: %d
%s LeafHash: %X
%s Aunts: %X
%s}`,
indent, sp.Total,
indent, sp.Index,
indent, sp.LeafHash,
indent, sp.Aunts,
indent)
}
```

증명 검증 단계별 로깅

```
func  debugVerifyProof(sp *Proof, rootHash []byte, leaf []byte) {
fmt.Printf("Verifying proof for index %d of %d total items\n", sp.Index, sp.Tota
fmt.Printf("Expected root: %X\n", rootHash)
fmt.Printf("Leaf data: %X\n", leaf)
fmt.Printf("Leaf hash: %X\n", sp.LeafHash)
fmt.Printf("Aunts: %X\n", sp.Aunts)
```

```
// 단계별 해시 계산 로깅

hash := tmhash.New()

computedHash, _ := sp.computeRootHash(hash)

fmt.Printf("Computed root: %X\n", computedHash)

}
```

참고 자료

- RFC-6962: Certificate Transparency
- CometBFT Merkle 패키지 문서
- 상태 구조 문서
- 블록 저장소 구조
- 라이트 클라이언트 구조